InterFi NETWORK

@INTERFINETWORK

SMART CONTRACT SECURITY AUDIT OF

TAKEMICHI INU

SMART CONTRACT AUDIT | SOLIDITY DEVELOPMENT & TESTING | PROJECT EVALUATION

RELENTLESSLY SECURING THE PUBLIC BLOCKCHAIN

Audit Introduction

Auditing Firm	InterFi Network
Audit Architecture	InterFi Echelon Auditing Standard
Language	Solidity
Client Firm	Takemichi Inu
Website	https://takemichinu.com/
Telegram	https://t.me/Takemichi_Inu/
Twitter	https://twitter.com/TakemichiInu/
Report Date	May 30, 2022

About Takemichi Inu

Takemichi ($TAKE) is here with the goal of delivering one of the best launchpads for our holders to safely diversify their portfolios without worrying about scams.

Audit Summary

InterFi team has performed a line-by-line manual analysis and automated review of smart contracts. Smart contracts were analyzed mainly for common contract vulnerabilities, exploits, and manipulation hacks. According to the audit:

- Takemichi Inu's solidity source code has **LOW RISK SEVERITY**
- Takemichi Inu's smart contract has an **ACTIVE OWNERSHIP**
- Takemichi Inu's centralization risk correlated to the active owner is **MEDIUM**
- Important owner privileges – **ADD AND REMOVE REWARD TOKEN, DEPOSIT REWARD TOKEN, EMERGENCY WITHDRAW**

Be aware that smart contracts deployed on the blockchain aren't resistant to internal exploit, external vulnerability, or hack. For a detailed understanding of risk severity, source code vulnerability, exploitability, and audit disclaimer, kindly refer to the audit.

🚨 Contract address: **Not deployed**

⛓️ Blockchain: **Not chained**

✅ Verify the authenticity of this report on InterFi's GitHub: **https://github.com/interfinetwork**

Table Of Contents

Audit Information

Audit Scope 5

Echelon Audit Standard

Audit Methodology 6

Risk Classification 8

Centralization Risk 9

Smart Contract Risk Assessment

Static Analysis 10

Software Analysis 12

Manual Analysis 15

SWC Attacks 17

Risk Status & Radar Chart 19

Audit Summary

Auditor's Verdict 20

Legal Advisory

Important Disclaimer 21

About InterFi Network 22

InterFi
Smart Contract
Security Audit

Audit Scope

InterFi was consulted by Takemichi Inu to conduct the smart contract security audit of their solidity source codes. The audit scope of work is strictly limited to the mentioned solidity file(s) only:

- TakemichiInu.sol

Solidity Source Code On InterFi GitHub

https://github.com/interfinetwork/audited-codes/blob/main/TakemichiInu.sol

Payment Verification

Network: Ethereum Chain

Hash: 0x7912e154aebb7bb4d9c8bc142663cd92cbd5959ccae9ad9c3642942abc4125a7

SHA-1 Hash

Solidity source code is audited at hash #56f25b02c0df323f1bbc8621bb370a199d999f90

Audit Methodology

The scope of this report is to audit the smart contract source code of Takemichi Inu. InterFi has scanned contracts and reviewed codes for common vulnerabilities, exploits, hacks, and back-doors. Due to being out of scope, InterFi has not tested contracts on testnet to assess any functional flaws. Below is the list of commonly known smart contract vulnerabilities, exploits, and hacks:

Category	
Smart Contract Vulnerabilities	❖ Re-entrancy ❖ Unhandled Exceptions ❖ Transaction Order Dependency ❖ Integer Overflow ❖ Unrestricted Action ❖ Incorrect Inheritance Order ❖ Typographical Errors ❖ Requirement Violation ❖ Gas Limit and Loops
Source Code Review	❖ Deployment Consistency ❖ Repository Consistency ❖ Data Consistency ❖ Token Supply Manipulation ❖ Access Control and Authorization ❖ Operations Trail and Event Generation ❖ Assets Manipulation ❖ Ownership Control ❖ Liquidity Access

InterFi Smart Contract Security Audit

InterFi's Echelon Audit Standard

The aim of InterFi's "Echelon" standard is to analyze smart contracts and identify the vulnerabilities and the hacks. Kindly note, InterFi does not test smart contracts on testnet. It is recommended that smart contracts are thoroughly tested prior to the audit submission. Mentioned are the steps used by InterFi to audit smart contracts:

1. Solidity smart contract source code reviewal:
   - Review of the specifications, sources, and instructions provided to InterFi to make sure we understand the size, and scope of the smart contract audit.
   - Manual review of code, which is the process of reading source code line-by-line to identify potential vulnerabilities.
2. Static, Manual, and Software analysis:
   - Test coverage analysis is the process of determining whether the test cases are covering the code and how much code is exercised when we run those test cases.
   - Symbolic execution is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarify, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, actionable recommendations to help you take steps to secure your smart contracts

Automated 3P frameworks used to assess the smart contract vulnerabilities

- Consensys Tools
- SWC Registry
- Solidity Coverage
- Open Zeppelin Code Analyzer
- Solidity Code Complier

Risk Classification

Smart contracts are generally designed to manipulate and hold funds denominated in ETH/BNB. This makes them very tempting attack targets, as a successful attack may allow the attacker to directly steal funds from the contract. Below are the typical risk levels of a smart contract:

Vulnerable: A contract is vulnerable if it has been flagged by a static analysis tool as such. As we will see later, this means that some contracts may be vulnerable because of a false positive.

Exploitable: A contract is exploitable if it is vulnerable and the vulnerability could be exploited by an external attacker. For example, if the "vulnerability" flagged by a tool is in a function that requires owning the contract, it would be vulnerable but not exploitable.

Exploited: A contract is exploited if it received a transaction on the main network which triggered one of its vulnerabilities. Therefore, a contract can be vulnerable or even exploitable without having been exploited.

InterFi
Smart Contract
Security Audit

Risk severity	Meaning
! High	This level vulnerabilities could be exploited easily and can lead to asset loss, data loss, asset, or data manipulation. They should be fixed right away.
! Medium	This level vulnerabilities are hard to exploit but very important to fix, they carry an elevated risk of smart contract manipulation, which can lead to high-risk severity
! Low	This level vulnerabilities should be fixed, as they carry an inherent risk of future exploits, and hacks which may or may not impact the smart contract execution.
! Informational	This level vulnerabilities can be ignored. They are code style violations and informational statements in the code. They may not affect the smart contract execution

Centralization Risk

Centralization risk is the most common cause of decentralized finance hacks. When a smart contract has an active contract ownership, the risk related to centralization is elevated. There are some well-intended reasons to be an active contract owner, such as:

- Contract owner can be granted the power to `pause()` or `lock()` the contract in case of an external attack.
- Contract owner can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale, and to list on an exchange.

Authorizing a full centralized power to a single body can be dangerous. Unfortunately, centralization related risks are higher than common smart contract vulnerabilities. Centralization of ownership creates a risk of rug pull scams, where owners cash out tokens in such quantities that they become valueless. **Most important question to ask here is, how to mitigate centralization risk?** Here's InterFi's recommendation to lower the risks related to centralization hacks:

- Smart contract owner's private key must be carefully secured to avoid any potential hack.
- Smart contract ownership should be shared by multi-signature (multi-sig) wallets.
- Smart contract ownership can be locked in a contract, user voting, or community DAO can be introduced to unlock the ownership.

Takemichi Inu's Centralization Status

- Takemichi Inu's smart contract has an **active ownership.**
- Smart contract is **not deployed** on blockchain at the time of the audit.

Static Analysis

Symbol	Meaning
🛑	Function can modify state
💵	Function is payable
🔒	Function is locked
🔐	Function can be accessed
❗	Important functionality

```
| **Context** | Implementation |  |||
| └ | _msgSender | Internal 🔒 |   | |
| └ | _msgData | Internal 🔒 |   | |
||||||
| **Ownable** | Implementation | Context |||
| └ | <Constructor> | Public ❗ | 🛑  |NO❗ |
| └ | owner | Public ❗ |   |NO❗ |
| └ | renounceOwnership | Public ❗ | 🛑  | onlyOwner |
| └ | transferOwnership | Public ❗ | 🛑  | onlyOwner |
| └ | _transferOwnership | Internal 🔒 | 🛑  | |
||||||
| **IERC20** | Interface |  |||
| └ | totalSupply | External ❗ |   |NO❗ |
| └ | balanceOf | External ❗ |   |NO❗ |
| └ | decimals | External ❗ |   |NO❗ |
| └ | transfer | External ❗ | 🛑  |NO❗ |
| └ | allowance | External ❗ |   |NO❗ |
| └ | approve | External ❗ | 🛑  |NO❗ |
| └ | transferFrom | External ❗ | 🛑  |NO❗ |
||||||
| **Address** | Library |  |||
| └ | isContract | Internal 🔒 |   | |
| └ | sendValue | Internal 🔒 | 🛑  | |
| └ | functionCall | Internal 🔒 | 🛑  | |
| └ | functionCall | Internal 🔒 | 🛑  | |
| └ | functionCallWithValue | Internal 🔒 | 🛑  | |
| └ | functionCallWithValue | Internal 🔒 | 🛑  | |
| └ | functionStaticCall | Internal 🔒 |   | |
| └ | functionStaticCall | Internal 🔒 |   | |
| └ | functionDelegateCall | Internal 🔒 | 🛑  | |
| └ | functionDelegateCall | Internal 🔒 | 🛑  | |
```

```
| └ | verifyCallResult | Internal 🔒 |   | |
||||||
| **Math** | Library |  |||
| └ | max | Internal 🔒 |   | |
| └ | min | Internal 🔒 |   | |
| └ | average | Internal 🔒 |   | |
| └ | ceilDiv | Internal 🔒 |   | |
||||||
| **SafeERC20** | Library |  |||
| └ | safeTransfer | Internal 🔒 | 🛑  | |
| └ | safeTransferFrom | Internal 🔒 | 🛑  | |
| └ | safeApprove | Internal 🔒 | 🛑  | |
| └ | safeIncreaseAllowance | Internal 🔒 | 🛑  | |
| └ | safeDecreaseAllowance | Internal 🔒 | 🛑  | |
| └ | _callOptionalReturn | Private 🔐 | 🛑  | |
||||||
| **MultiRewardsStake** | Implementation | Ownable |||
| └ | <Constructor> | Public ❗ | 🛑  |NO❗ |
| └ | totalSupply | External ❗ |   |NO❗ |
| └ | balanceOf | External ❗ |   |NO❗ |
| └ | lastTimeRewardApplicable | Public ❗ |   |NO❗ |
| └ | totalRewardTokens | External ❗ |   |NO❗ |
| └ | rewardPerToken | Public ❗ |   |NO❗ |
| └ | rewardForToken | External ❗ |   |NO❗ |
| └ | _rewardPerTokenStored | Private 🔐 |   | |
| └ | getRewardTokens | External ❗ |   |NO❗ |
| └ | earned | External ❗ |   |NO❗ |
| └ | _earned | Private 🔐 |   | |
| └ | getRewardForDuration | External ❗ |   |NO❗ |
| └ | stake | External ❗ |  💵 | updateReward |
| └ | withdraw | Public ❗ |  💵 | updateReward |
| └ | getReward | Public ❗ |  💵 |NO❗ |
| └ | exit | External ❗ |  💵 |NO❗ |
| └ | depositRewardTokens | External ❗ |  💵 | onlyOwner |
| └ | notifyRewardAmount | Public ❗ |  💵 | onlyOwner |
| └ | addRewardToken | External ❗ |  💵 | onlyOwner |
| └ | removeRewardToken | Public ❗ |  💵 | onlyOwner updateReward |
| └ | emergencyWithdrawal | External ❗ |  💵 | onlyOwner updateReward |
| └ | _updateReward | Private 🔐 | 🛑  | |
```

Software Analysis

Function Signatures

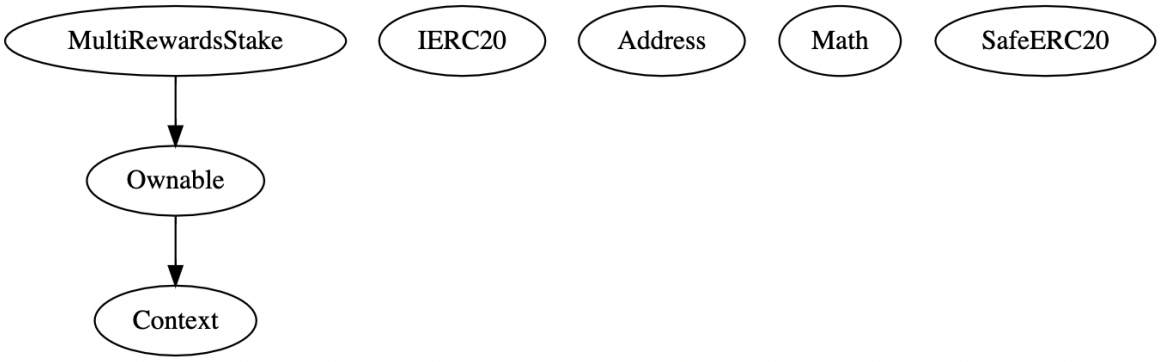
```
16279055  =>  isContract(address)
119df25f  =>  _msgSender()
8b49d47e  =>  _msgData()
8da5cb5b  =>  owner()
715018a6  =>  renounceOwnership()
f2fde38b  =>  transferOwnership(address)
d29d44ee  =>  _transferOwnership(address)
18160ddd  =>  totalSupply()
70a08231  =>  balanceOf(address)
313ce567  =>  decimals()
a9059cbb  =>  transfer(address,uint256)
dd62ed3e  =>  allowance(address,address)
095ea7b3  =>  approve(address,uint256)
23b872dd  =>  transferFrom(address,address,uint256)
24a084df  =>  sendValue(address,uint256)
a0b5ffb0  =>  functionCall(address,bytes)
241b5886  =>  functionCall(address,bytes,string)
2a011594  =>  functionCallWithValue(address,bytes,uint256)
d525ab8a  =>  functionCallWithValue(address,bytes,uint256,string)
c21d36f3  =>  functionStaticCall(address,bytes)
dbc40fb9  =>  functionStaticCall(address,bytes,string)
ee33b7e2  =>  functionDelegateCall(address,bytes)
57387df0  =>  functionDelegateCall(address,bytes,string)
946b5793  =>  verifyCallResult(bool,bytes,string)
6d5433e6  =>  max(uint256,uint256)
7ae2b5c7  =>  min(uint256,uint256)
2b7423ab  =>  average(uint256,uint256)
9cb35327  =>  ceilDiv(uint256,uint256)
d0c407e1  =>  safeTransfer(IERC20,address,uint256)
5beae096  =>  safeTransferFrom(IERC20,address,address,uint256)
d6dcec8d  =>  safeApprove(IERC20,address,uint256)
390cc046  =>  safeIncreaseAllowance(IERC20,address,uint256)
5164ffed  =>  safeDecreaseAllowance(IERC20,address,uint256)
becc5a20  =>  _callOptionalReturn(IERC20,bytes)
80faa57d  =>  lastTimeRewardApplicable()
1b0875a6  =>  totalRewardTokens()
cd3daf9d  =>  rewardPerToken()
a54f5e0d  =>  rewardForToken(address)
7aa7365a  =>  _rewardPerTokenStored(uint256)
c4f59f9b  =>  getRewardTokens()
008cc262  =>  earned(address)
df66c8f8  =>  _earned(address,uint256)
1c1f78eb  =>  getRewardForDuration()
a694fc3a  =>  stake(uint256)
2e1a7d4d  =>  withdraw(uint256)
```

```
3d18b912  =>  getReward()
e9fad8ee  =>  exit()
6a1af3b7  =>  depositRewardTokens(uint256[])
e71e7f78  =>  notifyRewardAmount(uint256[])
1c03e6cc  =>  addRewardToken(address)
3d509c97  =>  removeRewardToken(address)
6e353a1d  =>  emergencyWithdrawal(address)
3538f54e  =>  _updateReward(address,uint256)
```

InterFi

Smart Contract
Security Audit

Inheritance Graph



InterFi
Smart Contract
Security Audit

Manual Analysis

Function	Description	Available	Status
Total Supply	provides information about the total token supply	Yes	**Passed**
Balance Of	provides account balance of the owner's account	Yes	**Passed**
Transfer	executes transfers of a specified number of tokens to a specified address	Yes	**Passed**
Approve	allow a spender to withdraw a set number of tokens from a specified account	Yes	**Passed**
Allowance	returns a set number of tokens from a spender to the owner	Yes	**Passed**
Transfer Ownership	executes transfer of contract ownership to a specified wallet	Yes	**Passed**
Renounce Ownership	executes transfer of contract ownership to a dead address	Yes	**Passed**

InterFi
Smart Contract
Security Audit

Notable Information

- Takemichi Inu's smart contract contains `functionCall()`, `functionStaticCall()`, and `functionDelegateCall()`. However, proxy opcode sequence isn't identified. Verify proxy here: https://bscscan.com/proxyContractChecker
- Smart contract utilizes `math()` function to avoid common smart contract vulnerabilities.

```
string private _name = "Takemichi Inu";
library Math {
function max(uint256 a, uint256 b) internal pure returns (uint256) {
    return a >= b ? a : b;
function min(uint256 a, uint256 b) internal pure returns (uint256) {
    return a < b ? a : b;
function average(uint256 a, uint256 b) internal pure returns (uint256) {
    // (a + b) / 2 can overflow.
    return (a & b) + (a ^ b) / 2;
function ceilDiv(uint256 a, uint256 b) internal pure returns (uint256) {
    // (a + b - 1) / b can overflow on addition, so we distribute.
    return a / b + (a % b == 0 ? 0 : 1);
```

- Smart contract does not utilize **re-entrancy guard** to prevent re-entrant calls.
- Smart contract utilizes **redundant code** for `transferOwnership()`. Ideal transfer ownership code should look be written like:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
```

- Smart contract owner can **emergency withdraw** tokens from smart contract.

```
function emergencyWithdrawal(address token) external payable onlyOwner updateReward(address(0))
    uint256 balance = IERC20(token).balanceOf(address(this));
    require(balance > 0, "Contract holds no tokens");
```

SWC Attacks

SWC ID	Description	Status
SWC-101	Integer Overflow and Underflow	**Passed**
SWC-102	Outdated Compiler Version	**! Informational**
SWC-103	Floating Pragma	**Passed**
SWC-104	Unchecked Call Return Value	**Passed**
SWC-105	Unprotected Ether Withdrawal	**Passed**
SWC-106	Unprotected SELF-DESTRUCT Instruction	**Passed**
SWC-107	Re-entrancy	**! Low**
SWC-108	State Variable Default Visibility	**Passed**
SWC-109	Uninitialized Storage Pointer	**Passed**
SWC-110	Assert Violation	**Passed**
SWC-111	Use of Deprecated Solidity Functions	**Passed**
SWC-112	Delegate Call to Untrusted Callee	**Passed**
SWC-113	DoS with Failed Call	**Passed**
SWC-114	Transaction Order Dependence	**Passed**
SWC-115	Authorization through tx.origin	**Passed**
SWC-116	Block values as a proxy for time	**Passed**
SWC-117	Signature Malleability	**Passed**
SWC-118	Incorrect Constructor Name	**Passed**

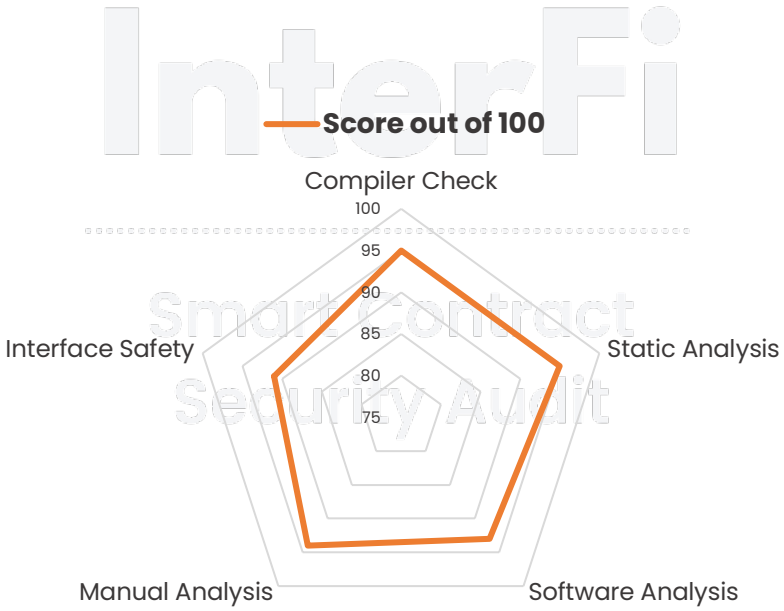
InterFi
Smart Contract
Security Audit

SWC-119	Shadowing State Variables	**Passed**
SWC-120	Weak Sources of Randomness from Chain Attributes	**Passed**
SWC-121	Missing Protection against Signature Replay Attacks	**Passed**
SWC-122	Lack of Proper Signature Verification	**Passed**
SWC-123	Requirement Violation	**Passed**
SWC-124	Write to Arbitrary Storage Location	**Passed**
SWC-125	Incorrect Inheritance Order	**Passed**
SWC-126	Insufficient Gas Griefing	**Passed**
SWC-127	Arbitrary Jump with Function Type Variable	**Passed**
SWC-128	DoS With Block Gas Limit	**Passed**
SWC-129	Typographical Error	**Passed**
SWC-130	Right-To-Left-Override control character (U+202E)	**Passed**
SWC-131	Presence of unused variables	**Passed**
SWC-132	Unexpected Ether balance	**Passed**
SWC-133	Hash Collisions With Multiple Variable Length Arguments	**Passed**
SWC-134	Message call with the hardcoded gas amount	**Passed**
SWC-135	Code With No Effects (Irrelevant/Dead Code)	**! Low**
SWC-136	Unencrypted Private Data On-Chain	**Passed**

InterFi
Smart Contract
Security Audit

Risk Status & Radar Chart

Risk Severity	Status
High	No high severity issues identified
Medium	No medium severity issues identified
Low	2 low severity issues identified
Informational	1 informational severity issue identified
Centralization Risk	Active contract ownership identified



Auditor's Verdict

InterFi team has performed a line-by-line manual analysis and automated review of smart contracts. Smart contracts were analyzed mainly for common contract vulnerabilities, exploits, and manipulation hacks. According to the audit:

- Takemichi Inu's smart contract source code has **LOW RISK SEVERITY**
- Takemichi Inu's smart contract has an **ACTIVE OWNERSHIP**
- Takemichi Inu's centralization risk correlated to the active owner is **MEDIUM**

Note for stakeholders

- Be aware that active smart contract owner privileges constitute an elevated impact on smart contract safety and security.
- If the smart contract is not deployed on any blockchain at the time of the audit, the contract can be modified or altered before blockchain development. Verify contract's deployment status in the audit report.
- Make sure that the project team's KYC/identity is verified by an independent firm.
- Always check if the contract's liquidity is locked. A longer liquidity lock plays an important role in the project's longevity. It is recommended to have multiple liquidity providers.
- Examine the unlocked token supply in the owner, developer, or team's private wallets. Understand the project's tokenomics, and make sure the tokens outside of the LP Pair are vested or locked for a longer period.

Important Disclaimer

InterFi Network provides contract development, testing, auditing and project evaluation services for blockchain projects. The purpose of the audit is to analyze the on-chain smart contract source code and to provide a basic overview of the project. **This report should not be transmitted, disclosed, referred to, or relied upon by any person for any purpose without InterFi's prior written consent.**

InterFi provides the easy-to-understand assessment of the project, and the smart contract (otherwise known as the source code). The audit makes no statements or warranties on the security of the code. It also cannot be considered as enough assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have used all the data at our disposal to provide the transparent analysis, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts. **Be aware that smart contracts deployed on a blockchain aren't resistant to external vulnerability, or a hack. Be aware that active smart contract owner privileges constitute an elevated impact on smart contract safety and security. Therefore, InterFi does not guarantee the explicit security of the audited smart contract.**

The analysis of the security is purely based on the smart contracts alone. No applications or operations were reviewed for security. No product code has been reviewed.

This report should not be considered as an endorsement or disapproval of any project or team. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. Do conduct your due diligence and consult your financial advisor before making any investment decisions.

About InterFi Network

InterFi Network provides intelligent blockchain solutions. InterFi is developing an ecosystem that is seamless and responsive. Some of our services: Blockchain Security, Token Launchpad, NFT Marketplace, etc. **InterFi's mission is to interconnect multiple services like Blockchain Security, DeFi, Gaming, and Marketplace under one ecosystem that is seamless, multi-chain compatible, scalable, secure, fast, responsive, and easy to use.**

InterFi is built by a decentralized team of UI experts, contributors, engineers, and enthusiasts from all over the world. Our team currently consists of 6+ core team members, and 10+ casual contributors. **InterFi provides manual, static, and automatic smart contract analysis, to ensure that project is checked against known attacks and potential vulnerabilities.**

To learn more, visit **https://interfi.network**

To view our audit portfolio, visit **https://github.com/interfinetwork**

To book an audit, message **https://t.me/interfiaudits**

InterFi
Smart Contract
Security Audit

@INTERFINETWORK

RELENTLESSLY SECURING THE PUBLIC BLOCKCHAIN | MADE IN CANADA